**Министерство науки и высшего образования Российской Федерации**
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
**НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

# ОТЧЕТ

# ПО ЛАБОРАТОРНОЙ РАБОТЕ №6
# «Работа с БД в СУБД MongoDB»

## по дисциплине «Проектирование и реализация баз данных»

**Обучающийся** Преображенский Артемий Евгеньевич
**Факультет** прикладной информатики
**Группа** K3241
**Направление подготовки** 09.03.03 Прикладная информатика
**Образовательная программа** Мобильные и сетевые технологии 2023
**Преподаватель** Говорова Марина Михайловна

Санкт-Петербург
2025

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ 2
ВВЕДЕНИЕ 4
ОСНОВНАЯ ЧАСТЬ 7
2.1 ВСТАВКА ДОКУМЕНТОВ В КОЛЛЕКЦИЮ 7
Практическое задание 2.1.1 7

2.2 ВЫБОРКА ДАННЫХ ИЗ БД 9
Практическое задание 2.2.1 9
Практическое задание 2.2.2 11
Практическое задание 2.2.3 11
Практическое задание 2.2.4 12

2.3 ЛОГИЧЕСКИЕ ОПЕРАТОРЫ 12
Практическое задание 2.3.1 12
Практическое задание 2.3.2 13
Практическое задание 2.3.3 13
Практическое задание 2.3.4 14

3.1 ЗАПРОС К ВЛОЖЕННЫМ ОБЪЕКТАМ 14
Практическое задание 3.1.1 14
Практическое задание 3.1.2 16

3.2 АГРЕГИРОВАННЫЕ ЗАПРОСЫ 17
Практическое задание 3.2.1 17
Практическое задание 3.2.2 17
Практическое задание 3.2.3 17

3.3 РЕДАКТИРОВАНИЕ ДАННЫХ 18
Практическое задание 3.3.1 18
Практическое задание 3.3.2 18
Практическое задание 3.3.3 19
Практическое задание 3.3.4 20
Практическое задание 3.3.5 20
Практическое задание 3.3.6 21
Практическое задание 3.3.7 22
3.4 УДАЛЕНИЕ ДАННЫХ ИЗ КОЛЛЕКЦИИ 22

Практическое задание 3.4.1 22
4.1 ССЫЛКИ В БД 25
Практическое задание 4.1.1 25

4.2 НАСТРОЙКА ИНДЕКСОВ 27
Практическое задание 4.2.1 27

4.3 УПРАВЛЕНИЕ ИНДЕКСАМИ 27
Практическое задание 4.3.1 27

4.4 ПЛАН ЗАПРОСА 28
Практическое задание 4.4.1 28

ЗАКЛЮЧЕНИЕ 32

# ВВЕДЕНИЕ

*Цели:*

1) овладеть практическими навыками установки СУБД MongoDB;

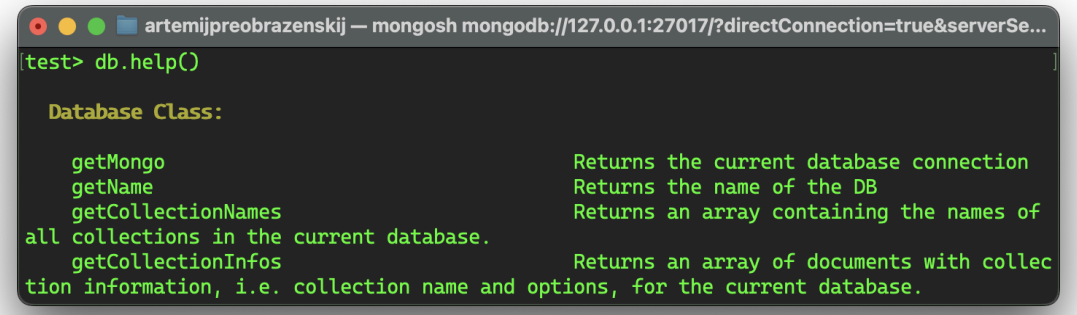
2) овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

*Оборудование:* компьютерный класс.

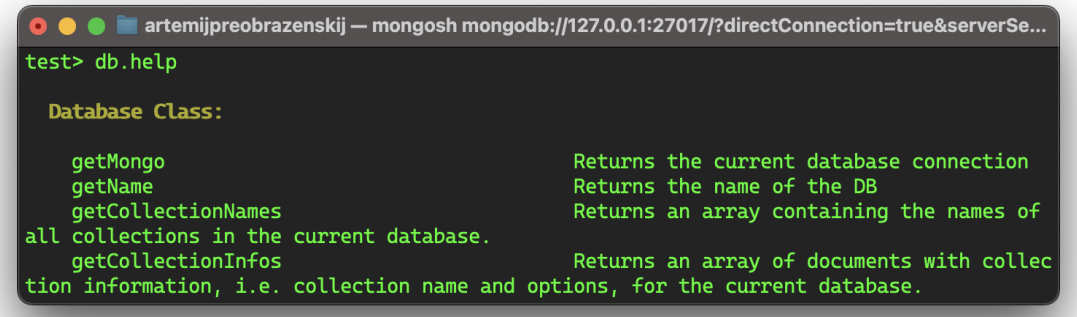
*Программное обеспечение:* СУБД MongoDB 4+, 8.0.4 (последняя)

В качестве предварительного знакомства с базовыми операциями MongoDB выполним практическое задание к лабораторной 6.1.

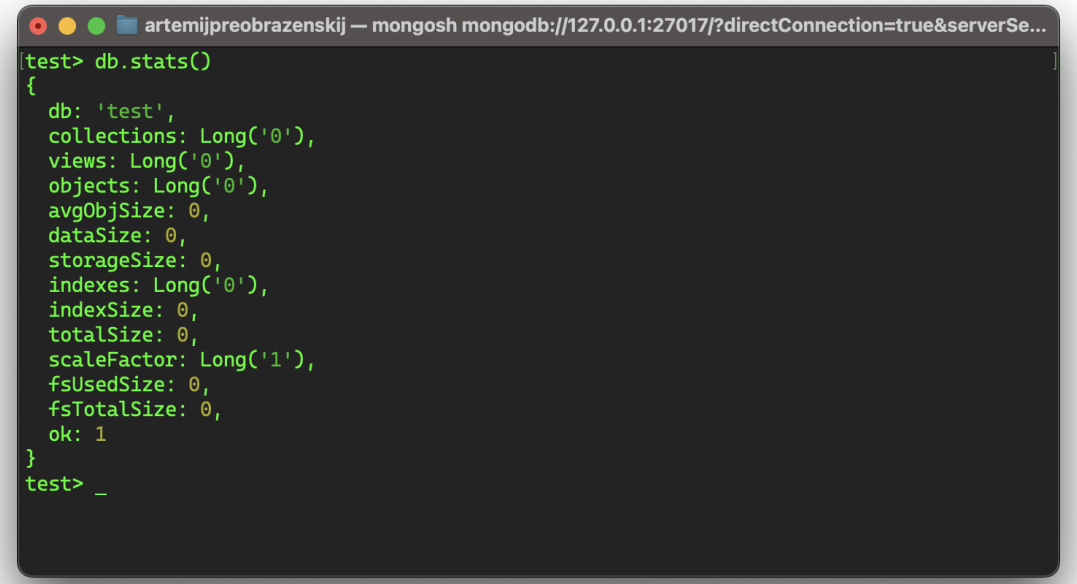
1. Установите MongoDB для обеих типов систем (32/64 бита).
2. Проверьте работоспособность системы запуском клиента mongo.
3. Выполните методы:
   a) db.help()




   b) db.help



c) db.stats()




4. Создайте БД learn.
5. Получите список доступных БД.
6. Создайте коллекцию unicorns, вставив в нее документ {name: 'Aurora', gender: 'f', weight: 450}.
7. Просмотрите список текущих коллекций.
8. Просмотрите статистику коллекции.



9. Переименуйте коллекцию unicorns.
10. Удалите коллекцию.
11. Удалите БД learn.



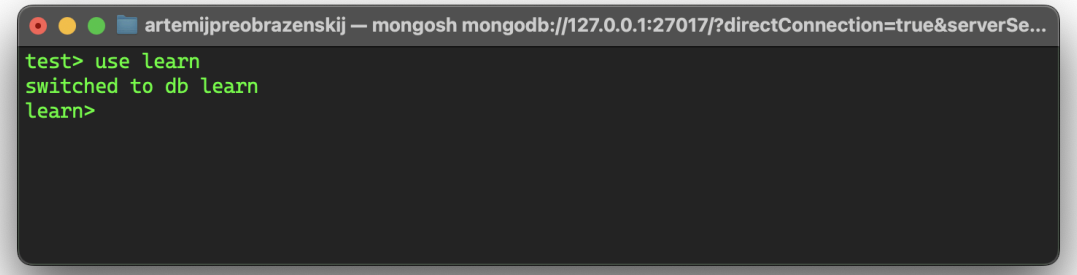
# ОСНОВНАЯ ЧАСТЬ

## 2.1 ВСТАВКА ДОКУМЕНТОВ В КОЛЛЕКЦИЮ

Практическое задание 2.1.1:

1) Создайте базу данных learn.

*use learn*

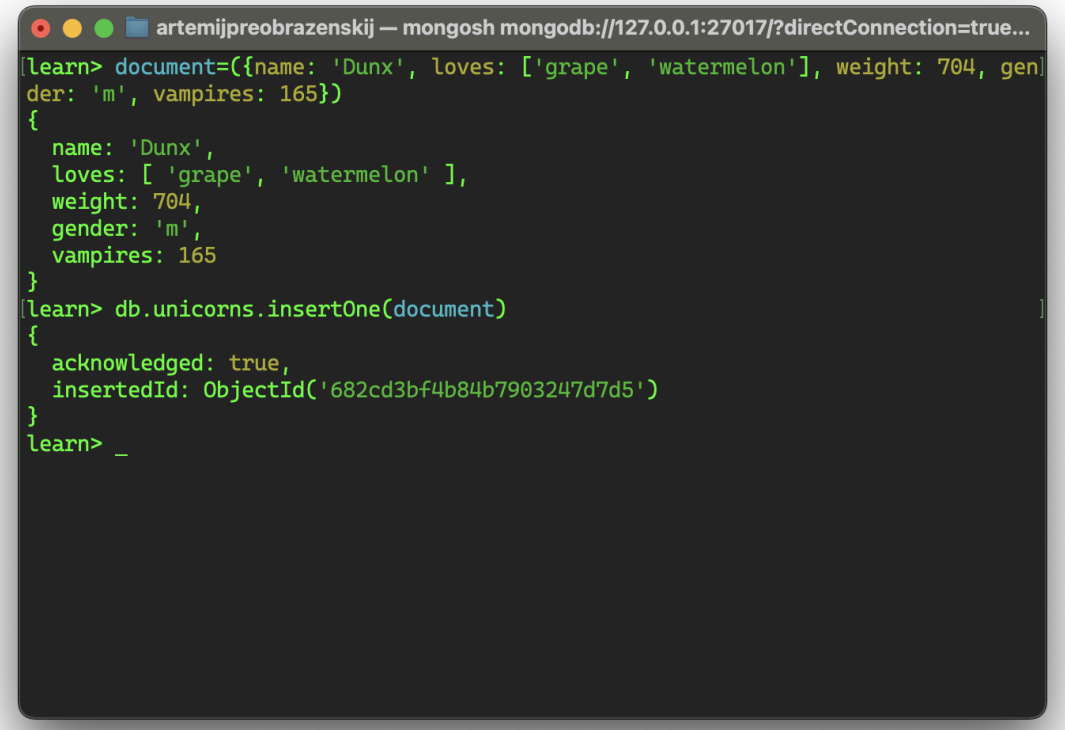



2) Заполните коллекцию единорогов unicorns:

*db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});*
*db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});*
*db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});*
*db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});*
*db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80});*
*db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});*
*db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});*
*db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});*
*db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});*
*db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});*
*db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});*




3) Используя второй способ, вставьте в коллекцию единорогов документ:
*document=({name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165})*
*db.unicorns.insertOne(document)*




4) Проверьте содержимое коллекции с помощью метода find.
*db.unicorns.find()*

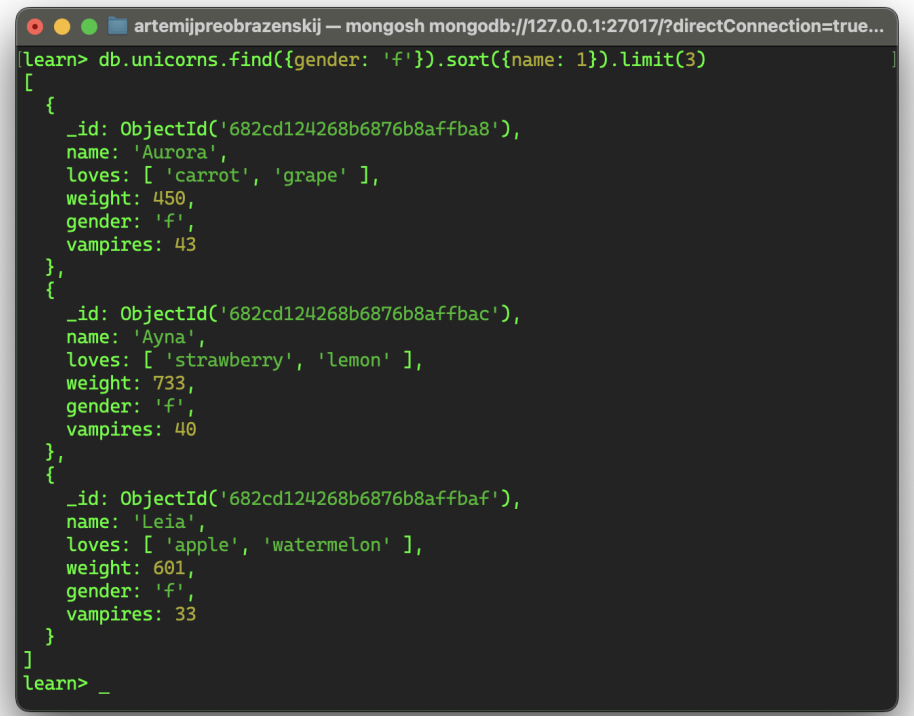



## 2.2 ВЫБОРКА ДАННЫХ ИЗ БД

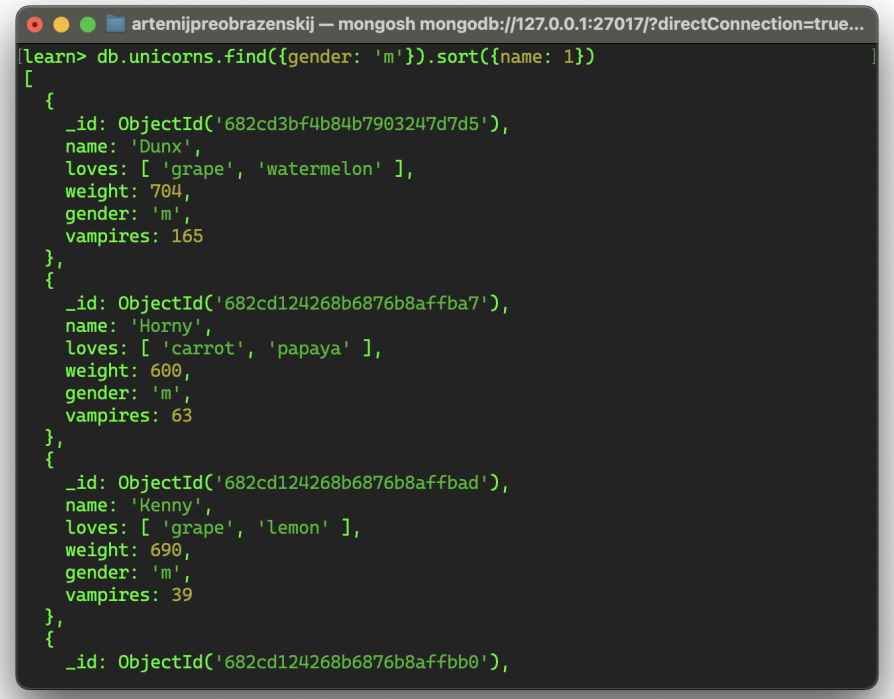
Практическое задание 2.2.1:

1) Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.

*db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)*



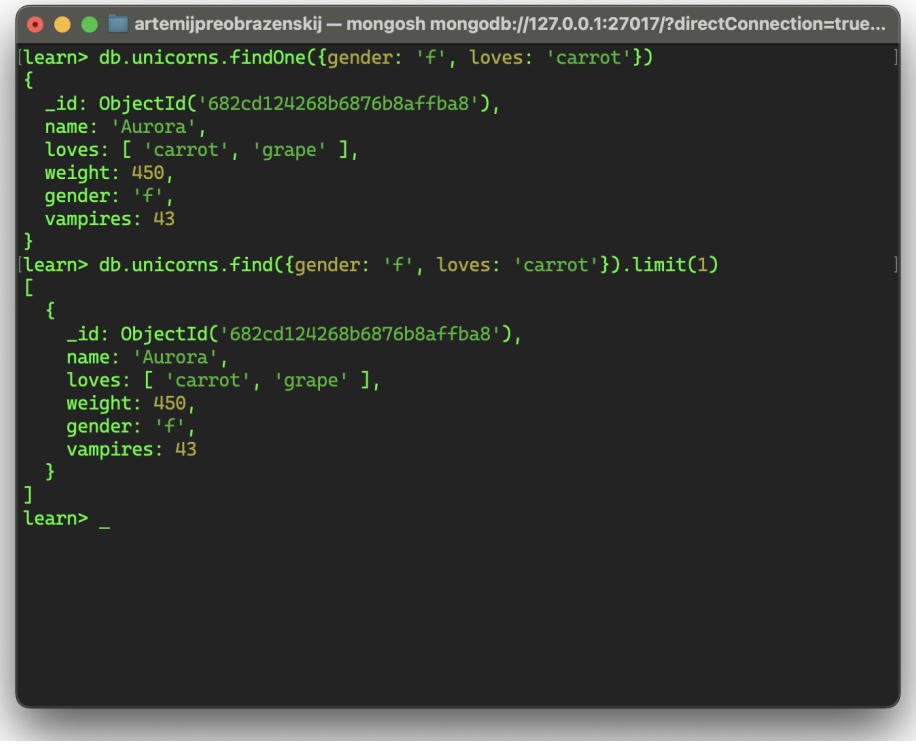
*db.unicorns.find({gender: 'm'}).sort({name: 1})*




2) Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций findOne и limit.

*db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)*

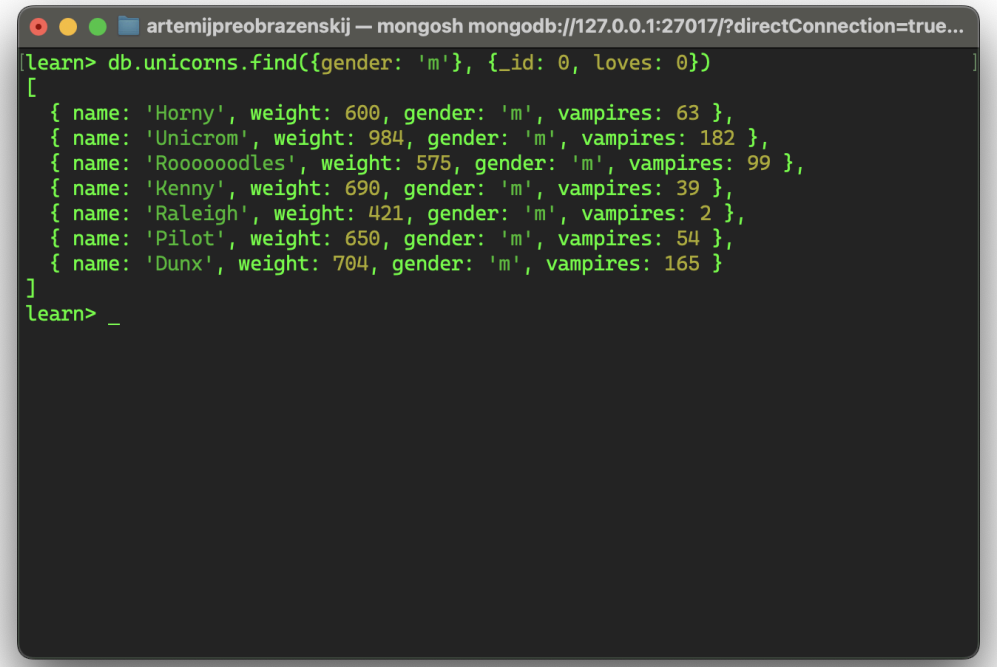
*db.unicorns.findOne({gender: 'f', loves: 'carrot'})*



Практическое задание 2.2.2:

1) Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

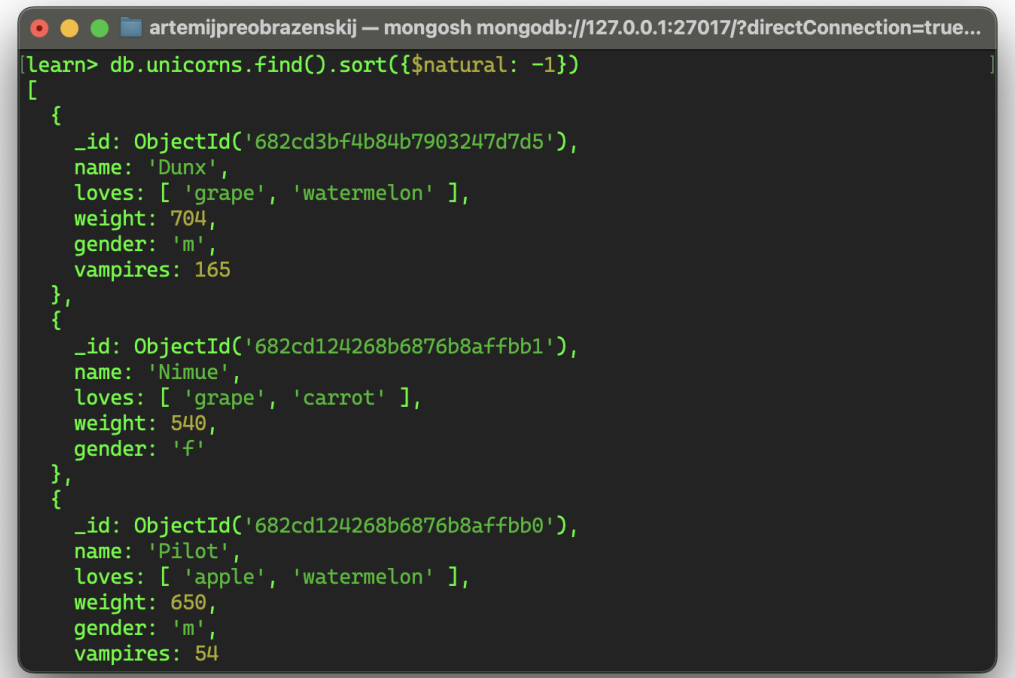
*db.unicorns.find({gender: 'm'}, {_id: 0, loves: 0})*




Практическое задание 2.2.3:l

1) Вывести список единорогов в обратном порядке добавления.

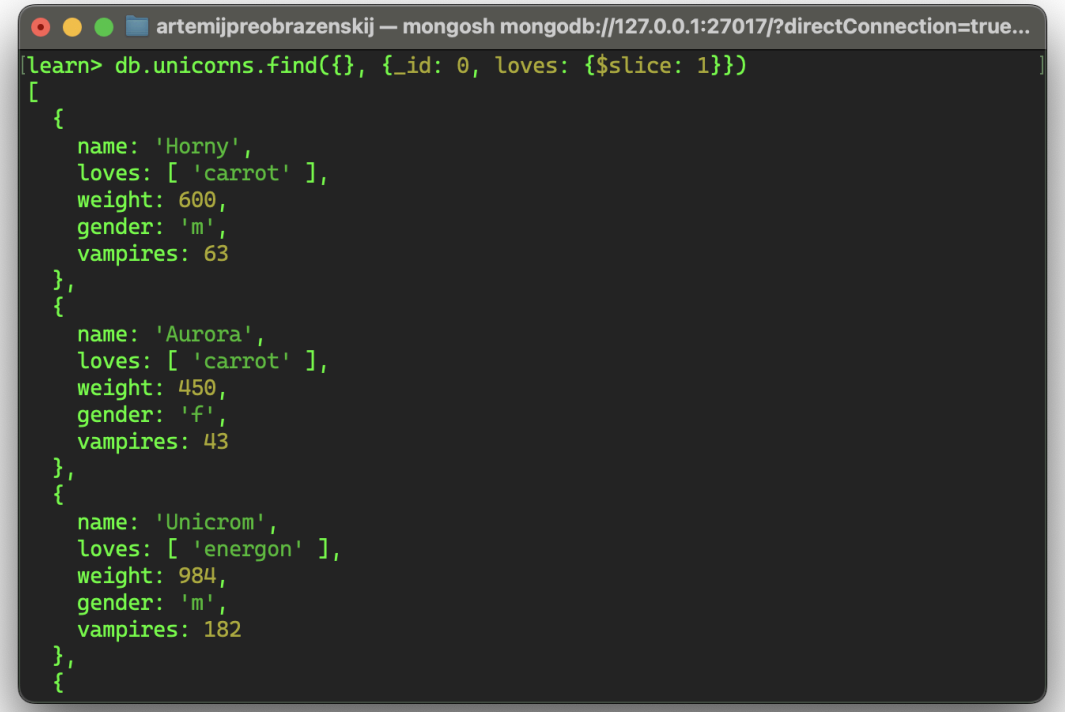
*db.unicorns.find().sort({$natural: -1})*



Практическое задание 2.1.4:

1) Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

*db.unicorns.find({}, {_id: 0, loves: {$slice: 1}})*

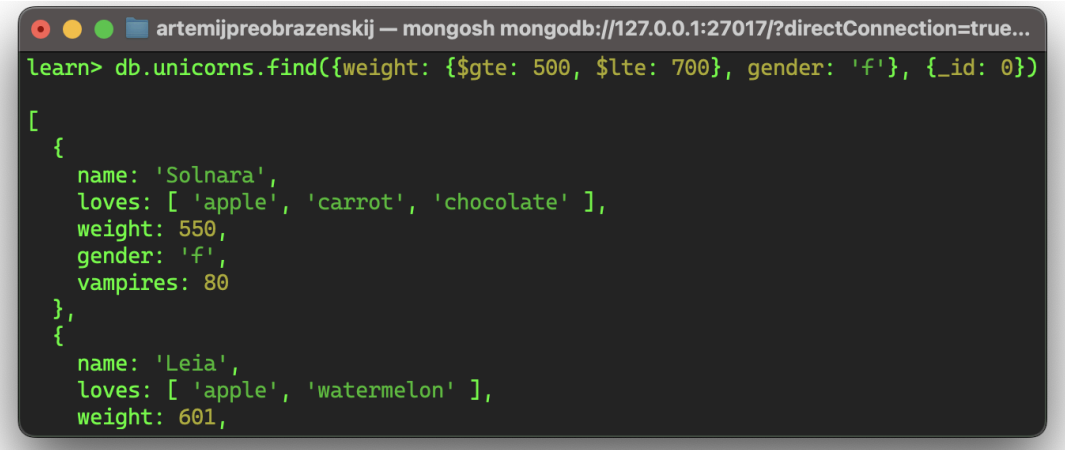



## 2.3 ЛОГИЧЕСКИЕ ОПЕРАТОРЫ

Практическое задание 2.3.1:

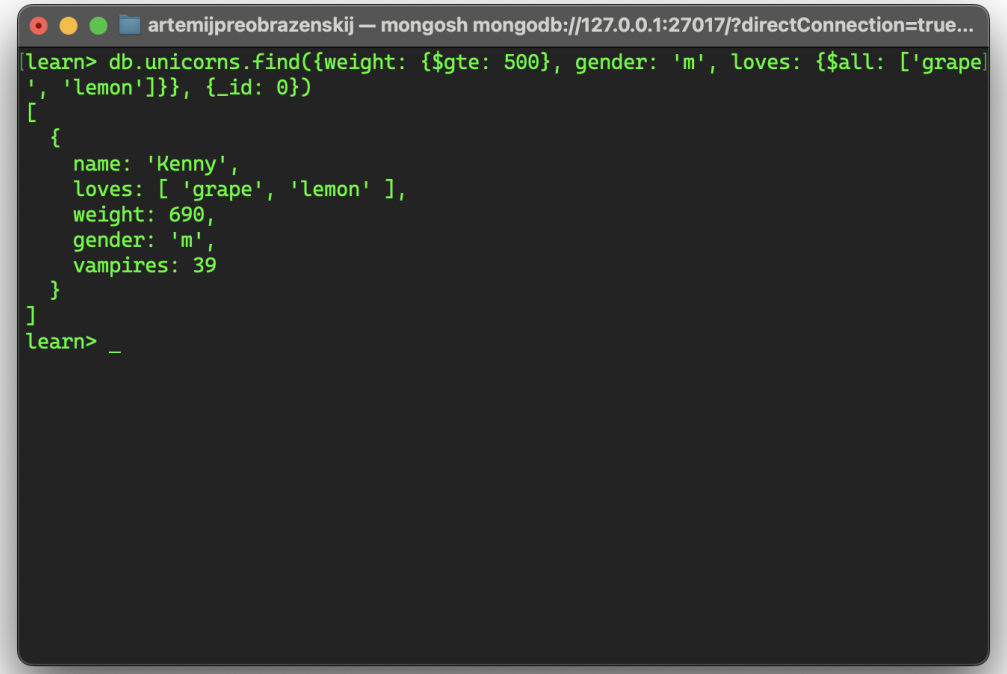
1) Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

*db.unicorns.find({weight: {$gte: 500, $lte: 700}, gender: 'f'}, {_id: 0})*



Практическое задание 2.3.2:

1) Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.
*db.unicorns.find({weight: {$gte: 500}, gender: 'm', loves: {$all: ['grape', 'lemon']}}, {_id: 0})*



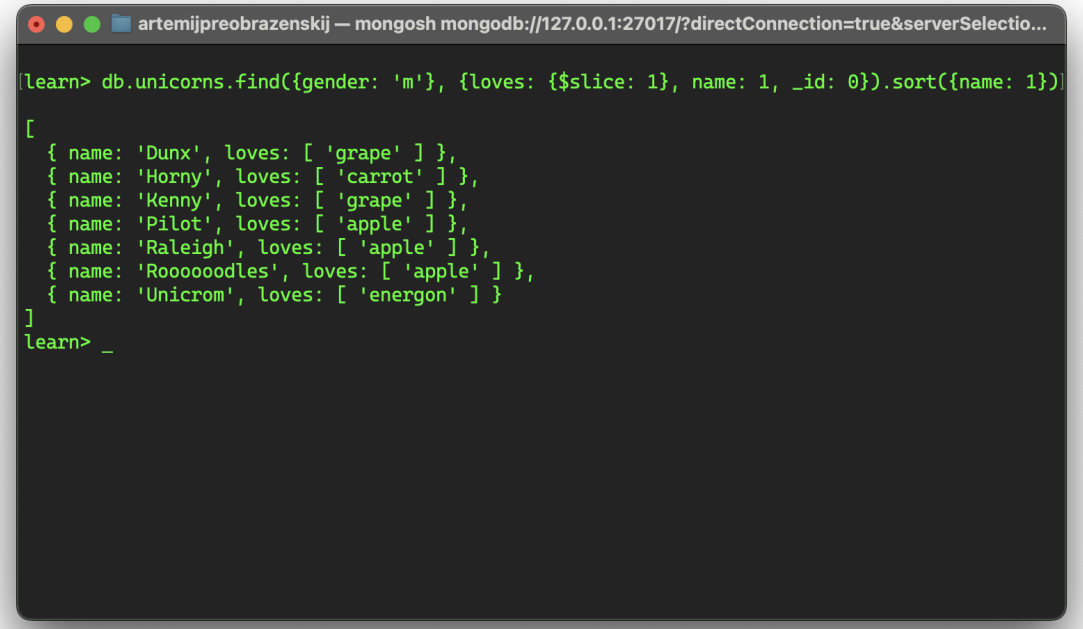

Практическое задание 2.3.3:

1) Найти всех единорогов, не имеющих ключ vampires.
*db.unicorns.find({vampires: null})*



Практическое задание 2.3.4:

1) Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.
*db.unicorns.find({gender: 'm'}, {loves: {$slice: 1}, name: 1, _id: 0}).sort({name: 1})*



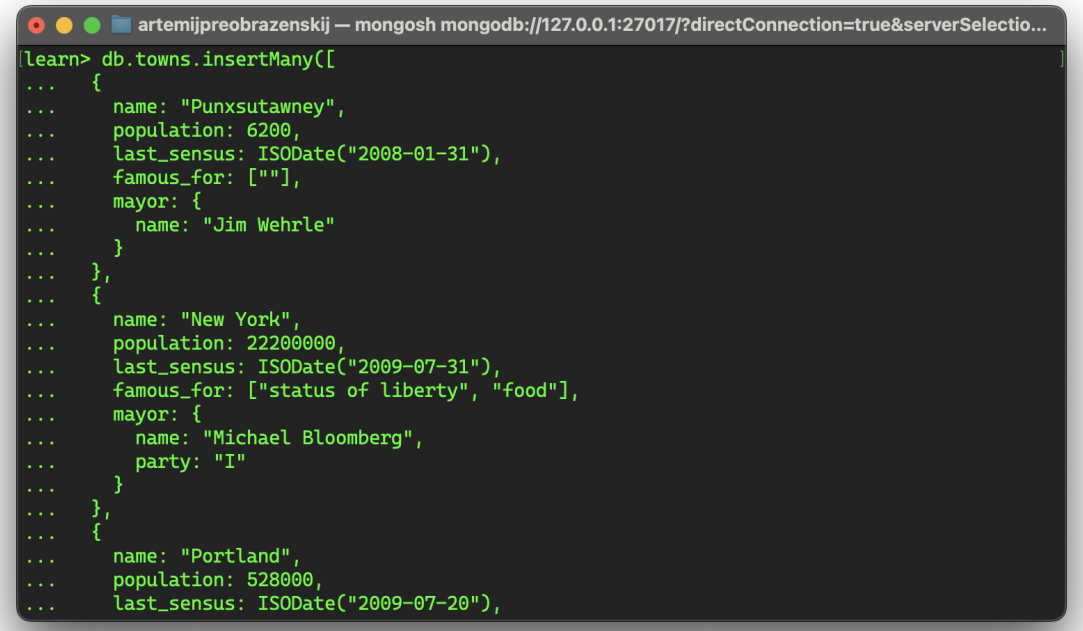

## 3.1 ЗАПРОС К ВЛОЖЕННЫМ ОБЪЕКТАМ

Практическое задание 3.1.1:

1) Создайте коллекцию towns, включающую следующие документы:

```
db.towns.insertMany([
  {
    name: "Punxsutawney",
    population: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: [""],
    mayor: {
      name: "Jim Wehrle"
    }
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
```

```
    party: "I"
   }
  },
  {
   name: "Portland",
   population: 528000,
   last_sensus: ISODate("2009-07-20"),
   famous_for: ["beer", "food"],
   mayor: {
    name: "Sam Adams",
    party: "D"
   }
  }
])
```




2) Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.

*db.towns.find({'mayor.party': 'I'}, {name: 1, mayor: 1, _id: 0})*



3) Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.

*db.towns.find({'mayor.party': null}, {name: 1, mayor: 1, _id: 0})*




Практическое задание 3.1.2:

1) Сформировать функцию для вывода списка самцов единорогов.

*fn = function() {return this.gender == 'm';}*



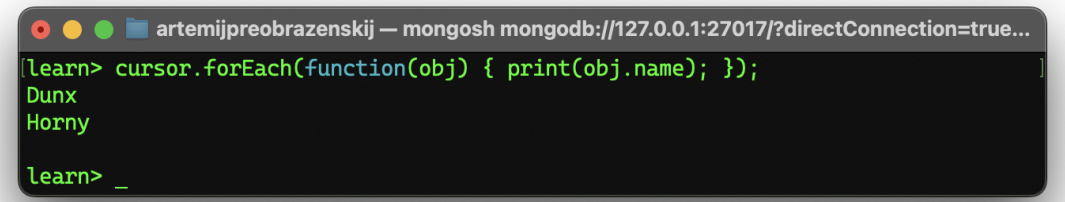

2) Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.

*var cursor = db.unicorns.find({$where: fn}).sort({name: 1}).limit(2)*




3) Вывести результат, используя forEach.

*cursor.forEach(function(obj) { print(obj.name); });*

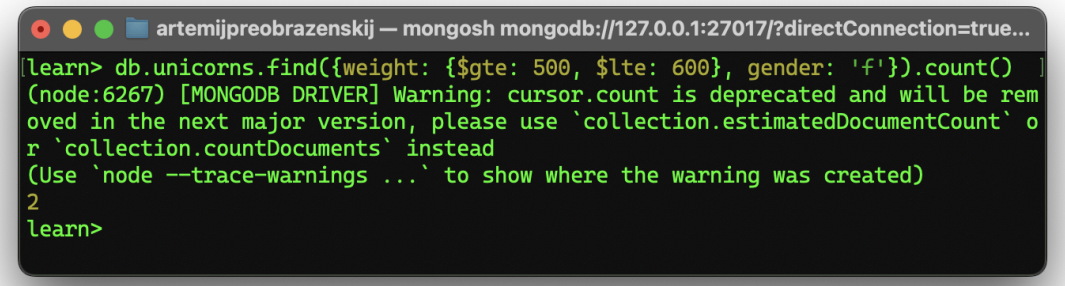


## 3.2 АГРЕГИРОВАННЫЕ ЗАПРОСЫ

Практическое задание 3.2.1:

1) Вывести количество самок единорогов весом от полутонны до 600 кг.

*db.unicorns.find({weight: {$gte: 500, $lte: 600}, gender: 'f'}).count()*




Практическое задание 3.2.2:

1) Вывести список предпочтений.

*db.unicorns.distinct('loves')*




Практическое задание 3.2.3:

1) Подсчитать количество особей единорогов обоих полов.

*db.unicorns.aggregate({'$group': {_id: '$gender', count: {$sum: 1}}})*



## 3.3 РЕДАКТИРОВАНИЕ ДАННЫХ

Практическое задание 3.3.1:

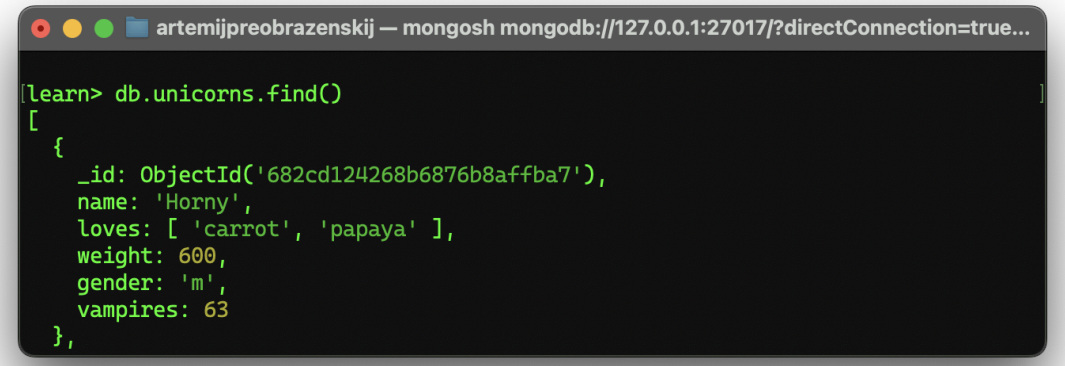
1. Выполнить команду:

*db.unicorns.insertOne({ name: 'Barny', loves: ['grape'], weight: 340, gender: 'm'})*




2. Проверить содержимое коллекции unicorns.

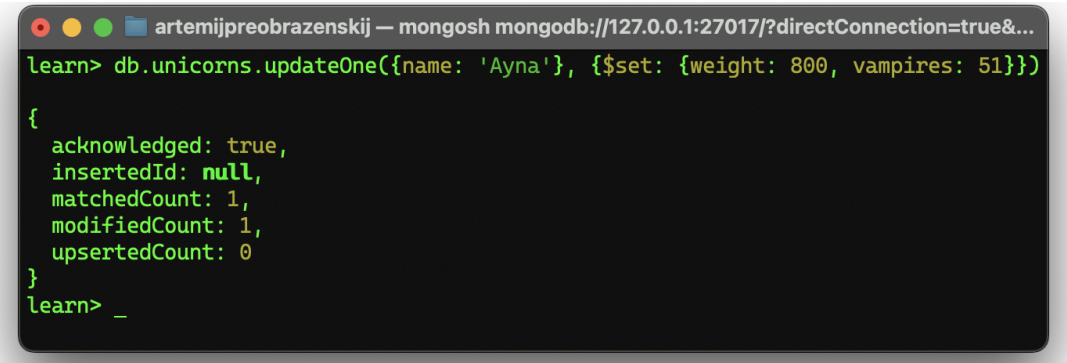
*db.unicorns.find()*




Практическое задание 3.3.2:

1. Для самки единорога Ayna внести изменения в БД: теперь ее вес 800, она убила 51 вампира.

*db.unicorns.updateOne({name: 'Ayna'}, {$set: {weight: 800, vampires: 51}})*



2. Проверить содержимое коллекции unicorns.

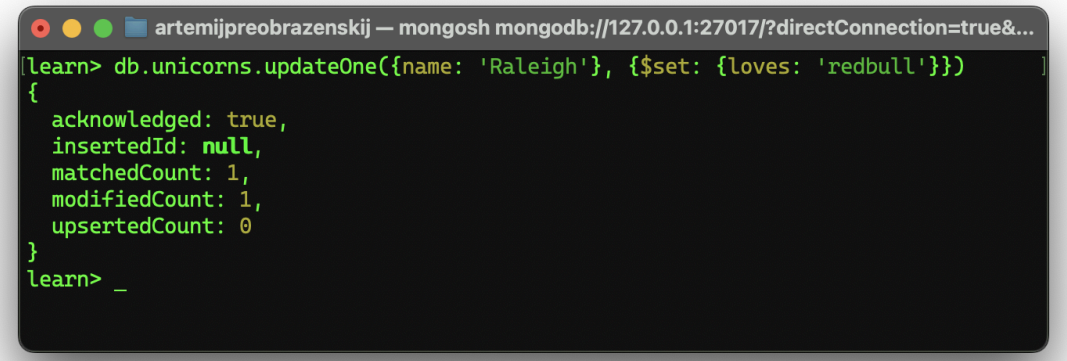
*db.unicorns.find({name: 'Ayna'})*




Практическое задание 3.3.3:

1. Для самца единорога Raleigh внести изменения в БД: теперь он любит ред булл.

*db.unicorns.updateOne({name: 'Raleigh'}, {$set: {loves: 'redbull'}})*




2. Проверить содержимое коллекции unicorns.

*db.unicorns.find({name: 'Raleigh'})*



Практическое задание 3.3.4:

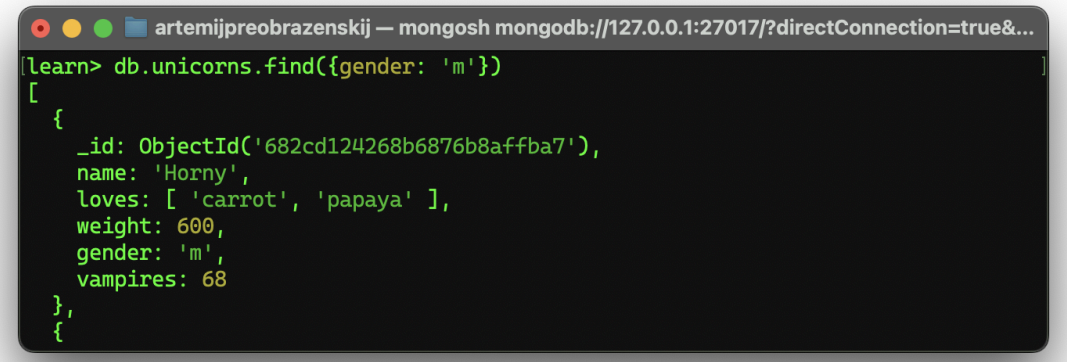
1. Всем самцам единорогов увеличить количество убитых вампиров на 5.

*db.unicorns.updateMany({gender: 'm'}, {$inc: {vampires:5}})*




2. Проверить содержимое коллекции unicorns.

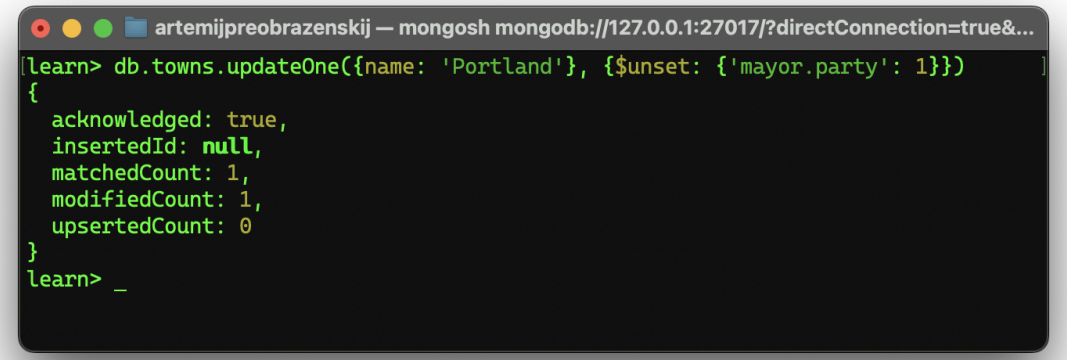
*db.unicorns.find({gender: 'm'})*




Практическое задание 3.3.5:

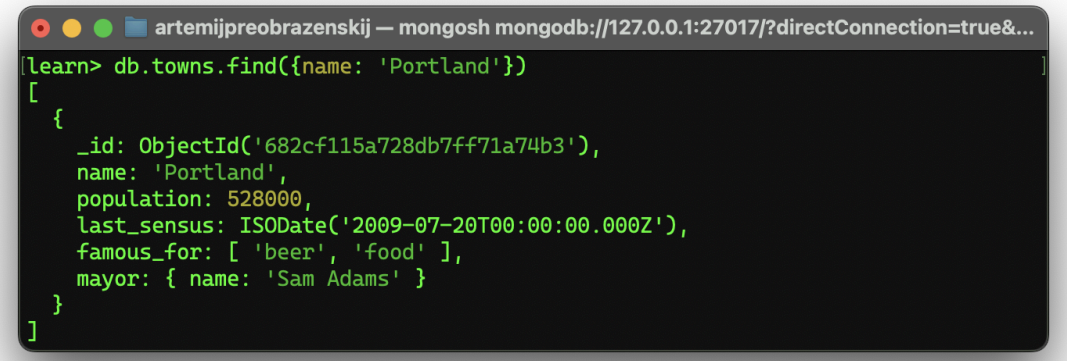
1. Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.

*db.towns.updateOne({name: 'Portland'}, {$unset: {'mayor.party': 1}})*



2. Проверить содержимое коллекции towns.

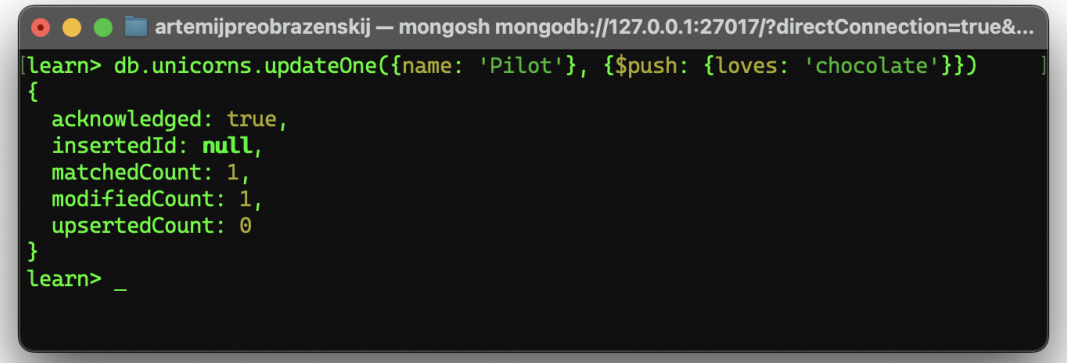
*db.towns.find({name: 'Portland'})*




Практическое задание 3.3.6:

1. Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.

*db.unicorns.updateOne({name: 'Pilot'}, {$push: {loves: 'chocolate'}})*




2. Проверить содержимое коллекции unicorns.

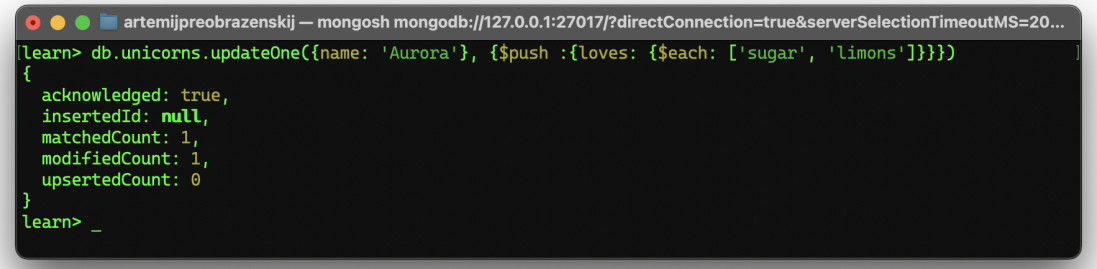
*db.unicorns.find({name: 'Pilot'})*



Практическое задание 3.3.7:

1. Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.

*db.unicorns.updateOne({name: 'Aurora'}, {$push :{loves: {$each: ['sugar', 'limons']}}})*




2. Проверить содержимое коллекции unicorns.

*db.unicorns.find({name: 'Aurora'})*



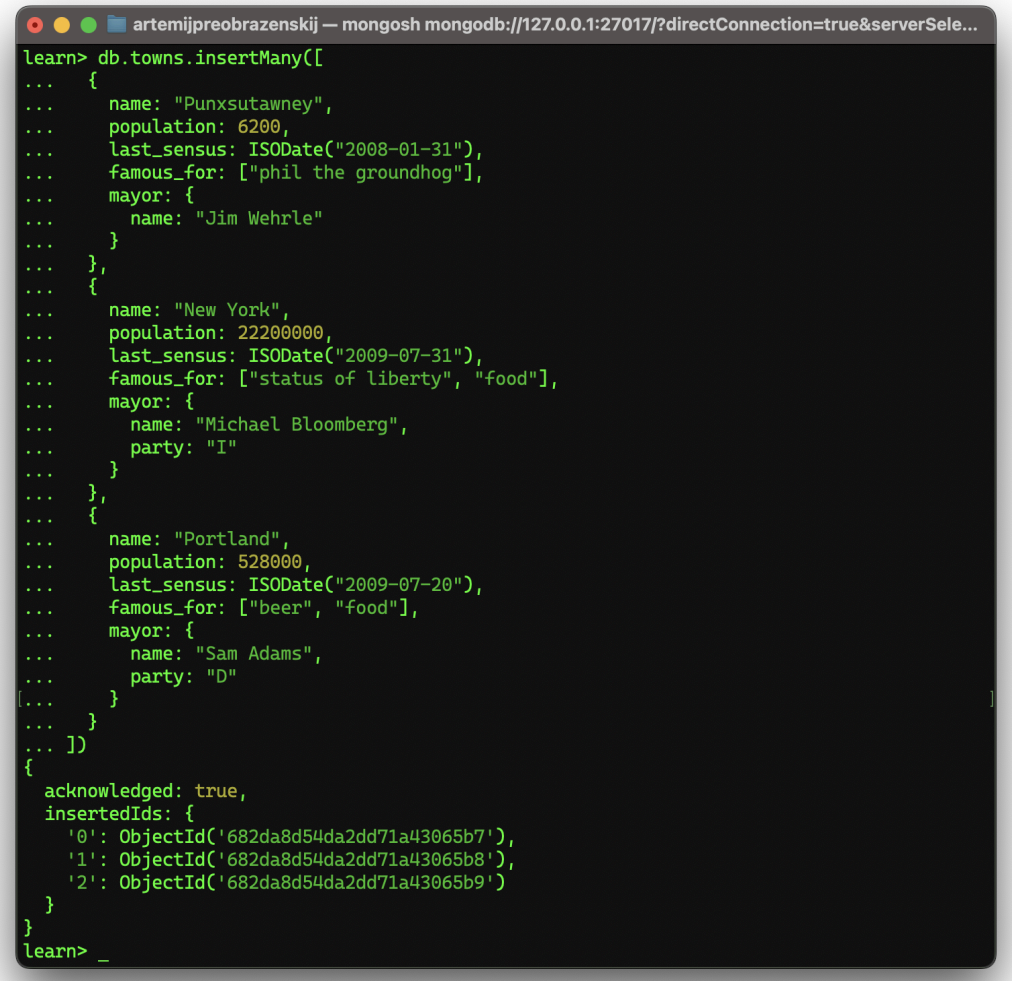

## 3.4 УДАЛЕНИЕ ДАННЫХ ИЗ КОЛЛЕКЦИИ

Практическое задание 3.4.1:

1) Создайте коллекцию towns, включающую следующие документы:

```
db.towns.insertMany([
 {
  name: "Punxsutawney",
  population: 6200,
  last_sensus: ISODate("2008-01-31"),
  famous_for: ["phil the groundhog"],
  mayor: {
   name: "Jim Wehrle"
  }
 },
 {
  name: "New York",
  population: 22200000,
```

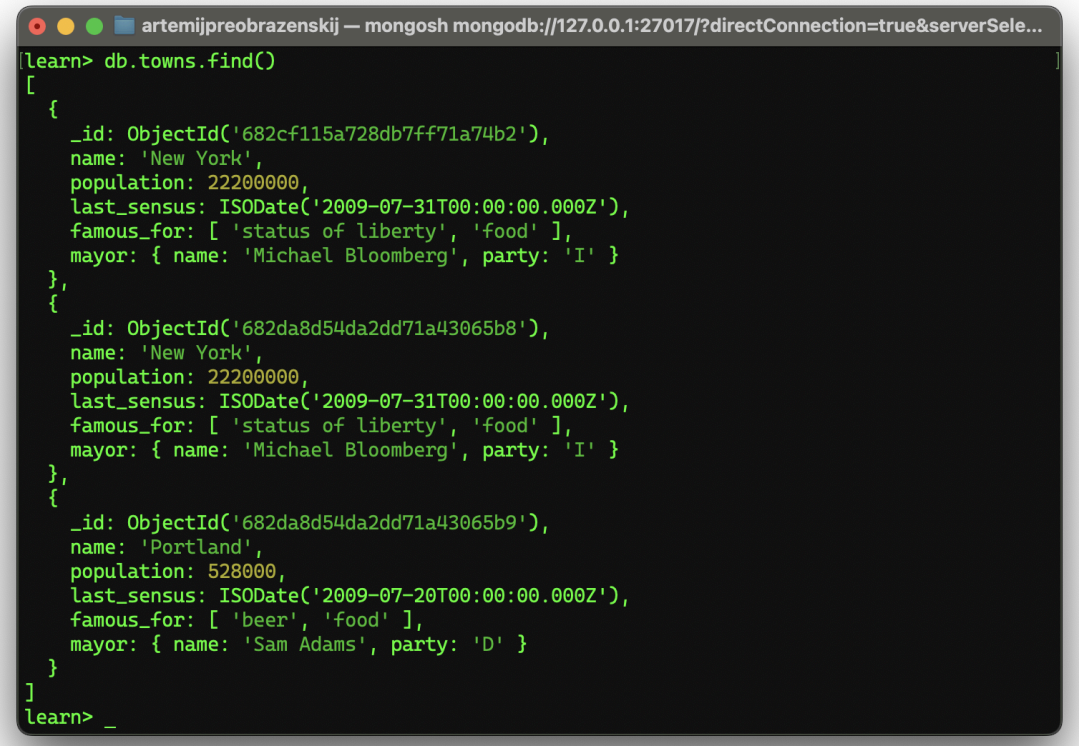
```
  last_sensus: ISODate("2009-07-31"),
  famous_for: ["status of liberty", "food"],
  mayor: {
    name: "Michael Bloomberg",
    party: "I"
  }
 },
 {
  name: "Portland",
  population: 528000,
  last_sensus: ISODate("2009-07-20"),
  famous_for: ["beer", "food"],
  mayor: {
    name: "Sam Adams",
    party: "D"
  }
 }
])
```



2) Удалите документы с беспартийными мэрами.
*db.towns.deleteMany({'mayor.party': null})*




3) Проверьте содержание коллекции.
*db.towns.find()*




4) Очистите коллекцию.
*db.towns.deleteMany({})*



5) Просмотрите список доступных коллекций.

*show collections*

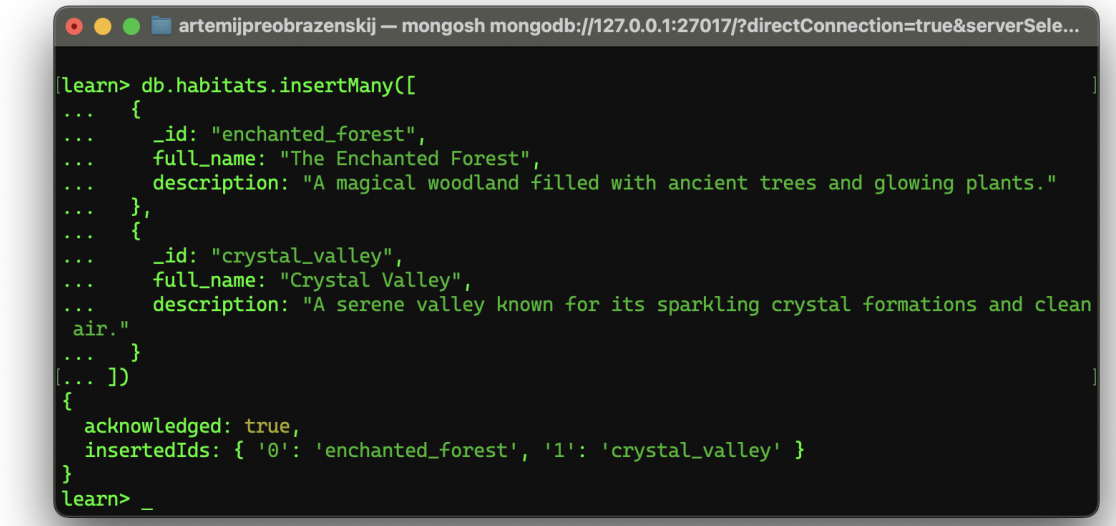



## 4.1 ССЫЛКИ В БД

Практическое задание 4.1.1:

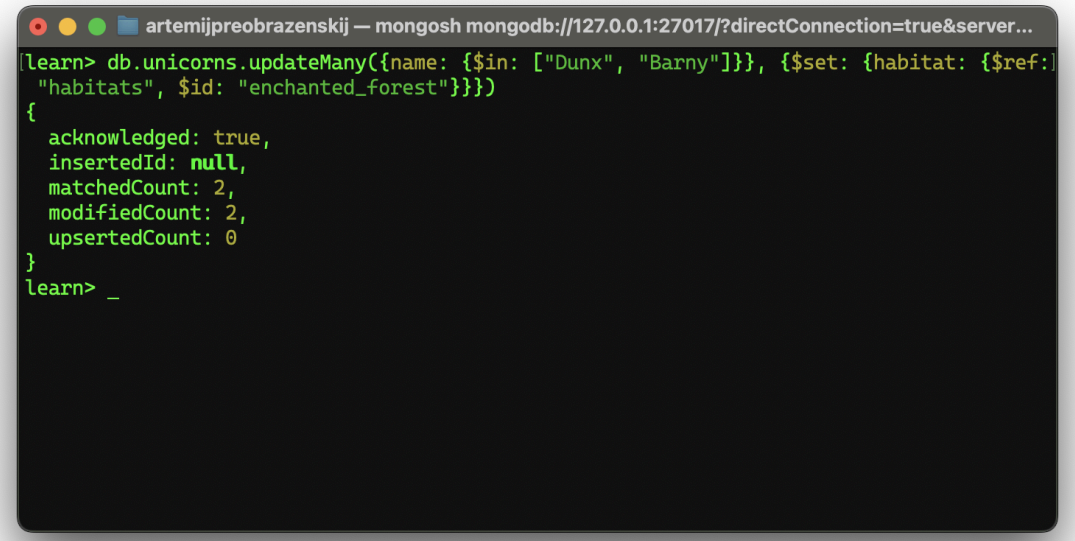
1) Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.

```
db.habitats.insertMany([
  {
    _id: "enchanted_forest",
    full_name: "The Enchanted Forest",
    description: "A magical woodland filled with ancient trees and glowing plants."
  },
  {
    _id: "crystal_valley",
    full_name: "Crystal Valley",
     description: "A serene valley known for its sparkling crystal formations and clean air."
  }
])
```



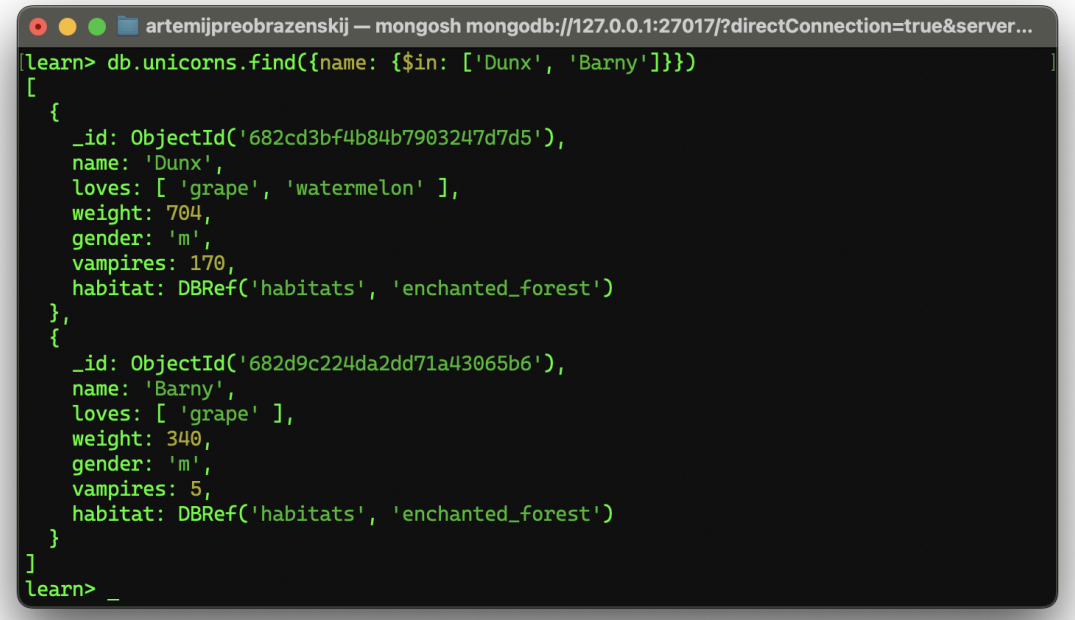
2) Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.

*db.unicorns.updateMany({name: {$in: ["Dunx", "Barny"]}}, {$set: {habitat: {$ref: "habitats", $id: "enchanted_forest"}}})*




3) Проверьте содержание коллекции единорогов.

*db.unicorns.find({name: {$in: ['Dunx', 'Barny']}})*



## 4.2 НАСТРОЙКА ИНДЕКСОВ

Практическое задание 4.2.1:

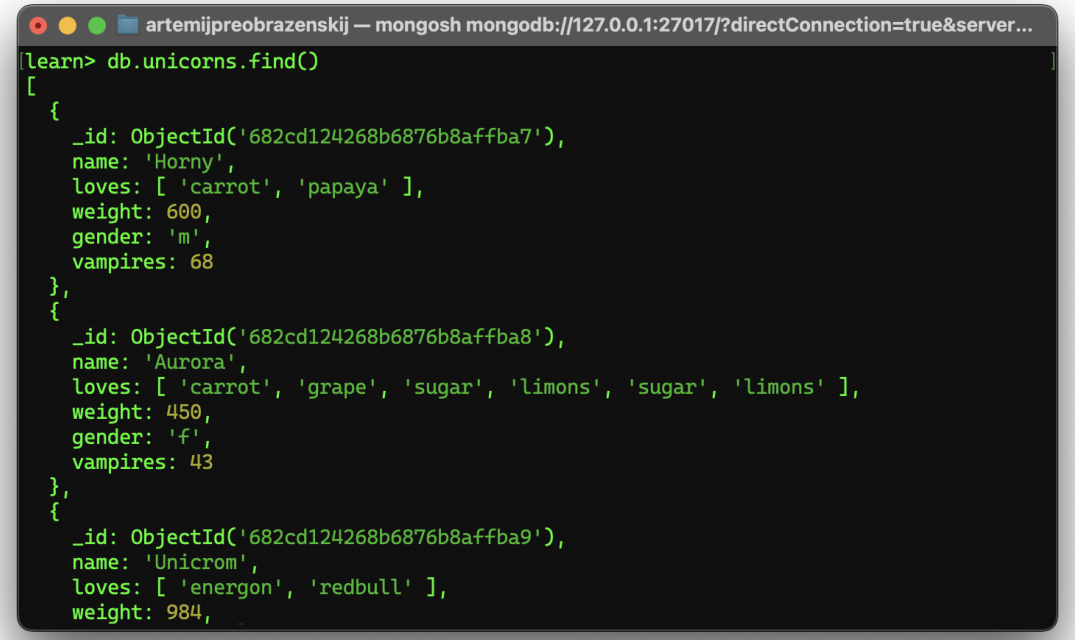
1. Проверьте, можно ли задать для коллекции unicorns индекс для ключа name с флагом unique

*db.unicorns.createIndex({"name" : 1}, {"unique" : true})*




2. Содержание коллекции единорогов unicorns:

*db.unicorns.find()*

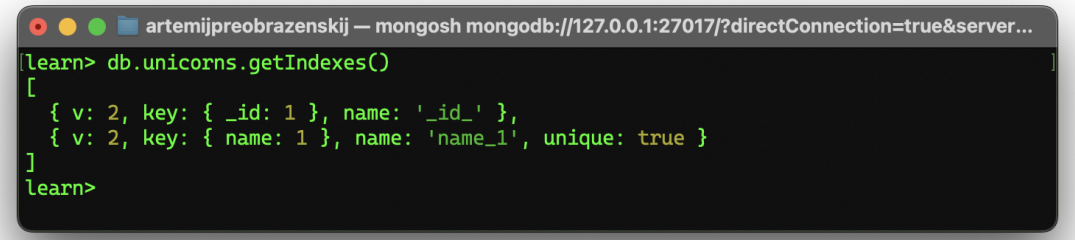



## 4.3 УПРАВЛЕНИЕ ИНДЕКСАМИ

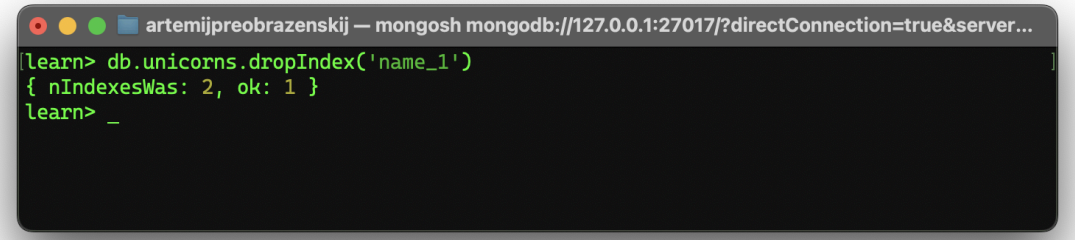
Практическое задание 4.3.1:

1) Получите информацию о всех индексах коллекции unicorns .

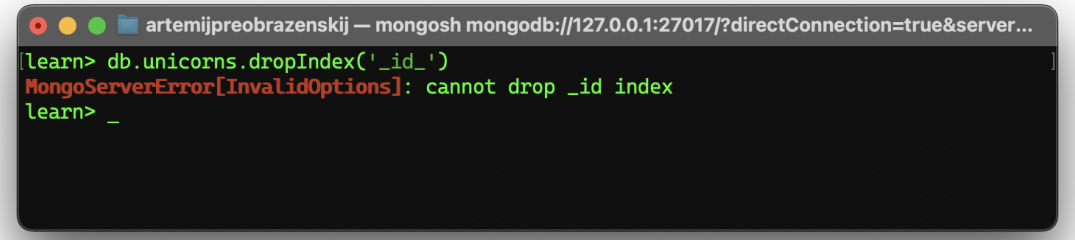
*db.unicorns.getIndexes()*




2) Удалите все индексы, кроме индекса для идентификатора.
*db.unicorns.dropIndex('name_1')*



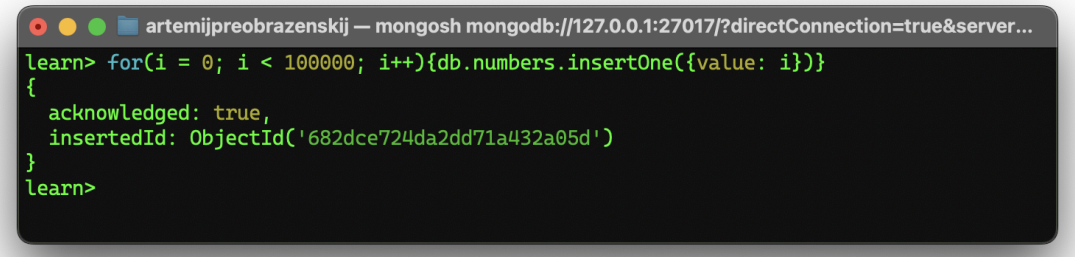

3) Попытайтесь удалить индекс для идентификатора.
*db.unicorns.dropIndex('_id_')*




## 4.4 ПЛАН ЗАПРОСА

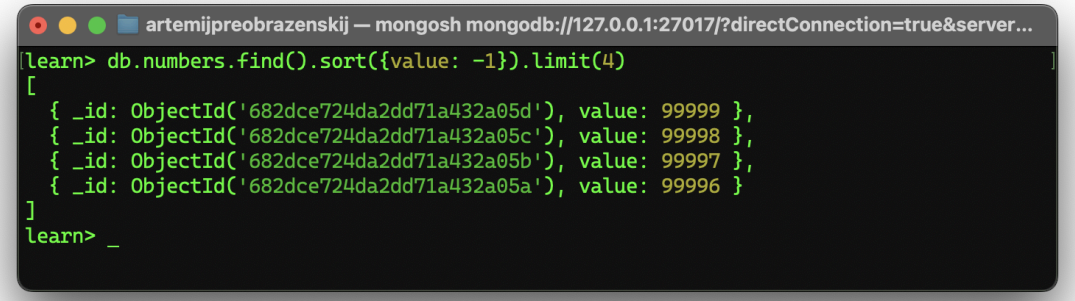
Практическое задание 4.4.1:

1) Создайте объемную коллекцию numbers, задействовав курсор:
*for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}*



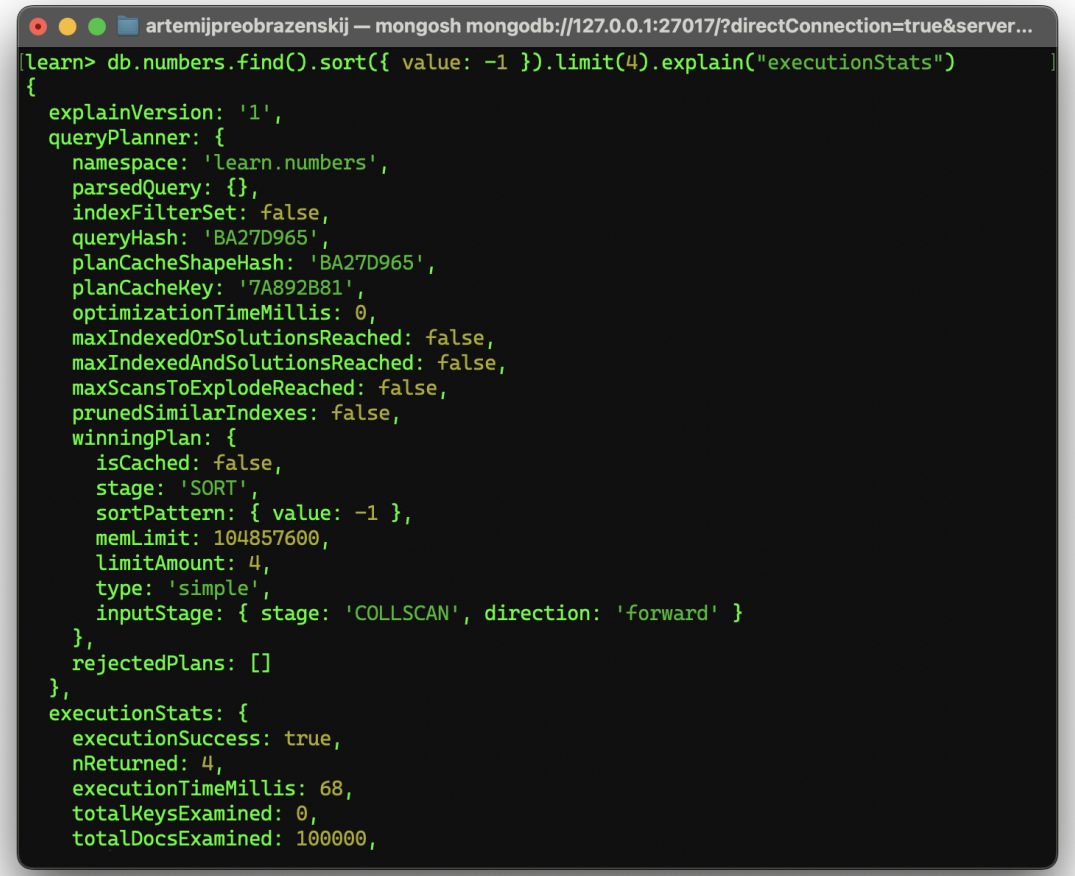
2) Выберите последних четыре документа.

*db.numbers.find().sort({value: -1}).limit(4)*




3) Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра executionTimeMillis)

*db.numbers.find().sort({ value: -1 }).limit(4).explain("executionStats")*




Вывод: время на выполнение запроса - 68 миллисекунд.

4) Создайте индекс для ключа value.

*db.numbers.createIndex({value: 1})*



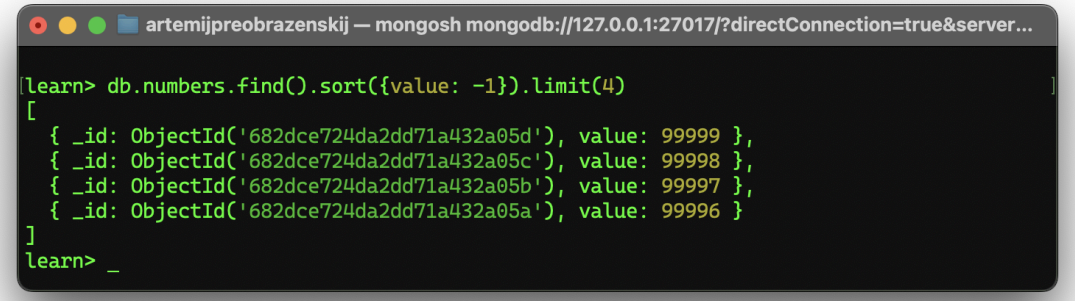

5) Получите информацию о всех индексах коллекции numbers.

*db.numbers.getIndexes()*




6) Выполните запрос 2.

*db.numbers.find().sort({value: -1}).limit(4)*




7) Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?

*db.numbers.find().sort({ value: -1 }).limit(4).explain("executionStats")*

Вывод: с установленным индексом вопрос выполняется мгновенно




8) Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

Вывод: Разумеется, запрос с индексом более эффективен, так как MongoDB использует B-дерево и позволяет избежать полного сканирования коллекции.

# ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были изучены и практически освоены основные операции работы с документно-ориентированной базой данных MongoDB. Были выполнены задания по вставке данных в коллекции с помощью insertOne и insertMany, выборке документов с использованием фильтрации по полям, логических и сравнительных операторов, применению проекций для выборочной загрузки полей документов, сортировке результатов запросов, а также обновлению данных с использованием различных операторов. Дополнительно была отработана процедура удаления документов по условиям, работа с вложенными структурами и массивами, включая фильтрацию по элементам массивов. Завершающим этапом стало создание и удаление индексов, а также анализ их влияния на производительность запросов. Подводя итог, можно сказать, что я получил ценный практический опыт по работе не реляционными бд.